YouTube API compliance audit - all evidence

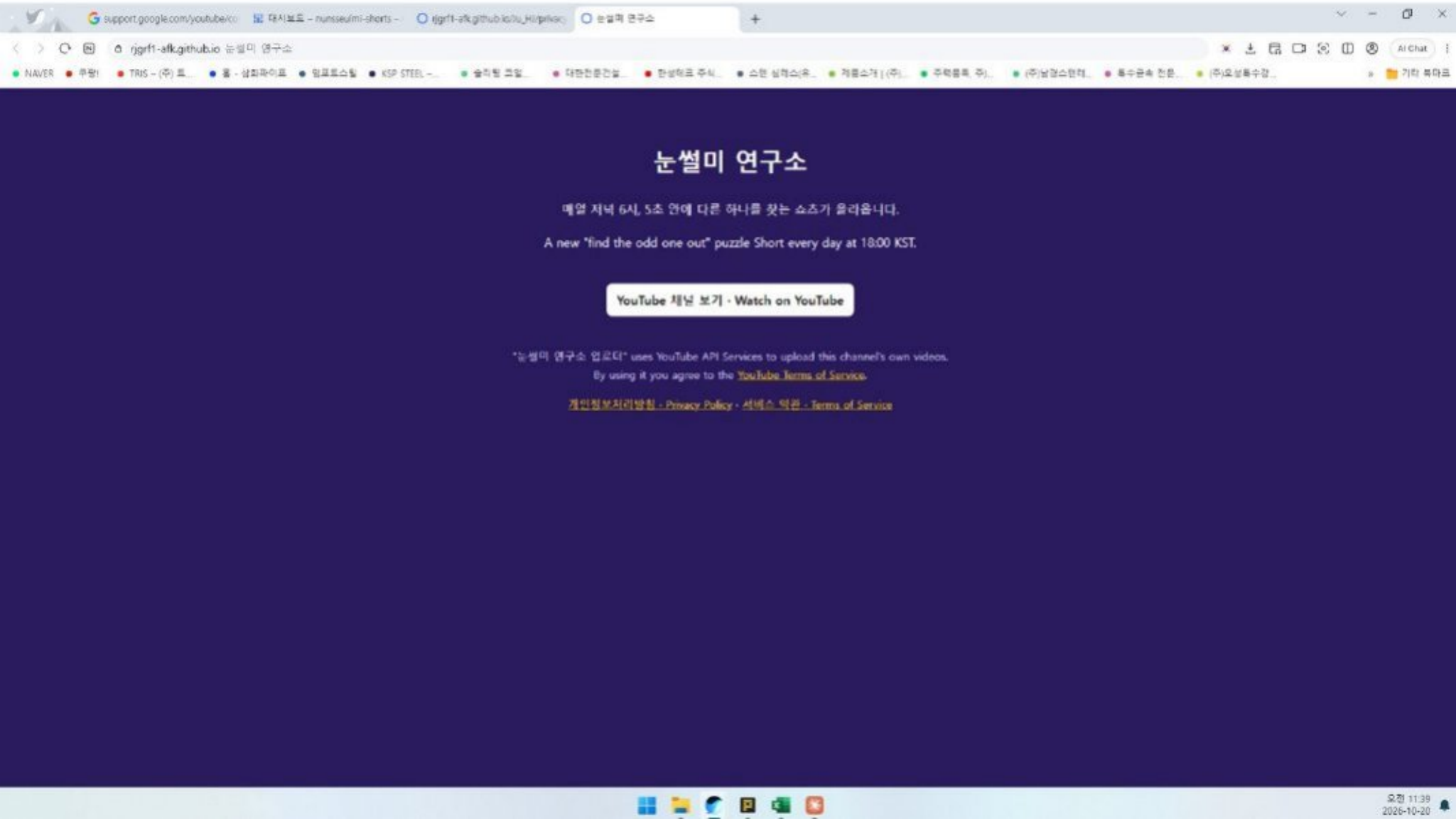
App: 눈썰미 연구소 업로더 · project 203046541838 · @nunsseulmi_lab

1. Privacy policy page (browser, with address bar)
2. Homepage with YouTube attribution and policy links
3. Terms of Service (https://rjgrf1-afk.github.io/Ju_HJ/terms.html)
4. Architecture diagram
5. User flow diagram
6. Conditional evidence: a) OAuth flow, b) upload, d) analytics

1. Privacy policy page



2. Homepage


rjgrf1-afk.github.io 눈썰미 연구소
NAVER
눈썰미 연구소
매일 저녁 6시, 5초 안에 다른 하나를 찾는 쇼츠가 올라옵니다.
A new "find the odd one out" puzzle Short every day at 18:00 KST.
YouTube 채널 보기 · Watch on YouTube
"눈썰미 연구소 업로더" uses YouTube API Services to upload this channel's own videos.
By using it you agree to the YouTube Terms of Service.
개인정보처리방침 · Privacy Policy · 서비스 약관 · Terms of Service

Terms of Service

눈썰미 연구소 업로더 (Nunsseulmi Lab Uploader) · Effective October 20, 2026

1. What this tool is

Nunsseulmi Lab Uploader ("the tool") is a private script operated by the owner of the YouTube channel @nunsseulmi_lab. It uploads and schedules the owner's own original Shorts and reads the owner's own channel statistics. It is not offered, sold, or licensed to anyone else.

2. YouTube API Services

The tool uses YouTube API Services. By using the tool you agree to be bound by the YouTube Terms of Service. The Google Privacy Policy also applies.

3. Permitted use

- The tool may only act on the channel of the person who authorized it with Google OAuth.
- Only original content that the channel owner has the rights to may be uploaded.
- The tool must not be used to collect data about viewers or other channels, to manipulate metrics, or to send spam.

4. Data and privacy

How data is accessed, stored, and deleted is described in the Privacy Policy. Access can be revoked at any time at myaccount.google.com/permissions.

5. No warranty

The tool is provided as is, without warranty of any kind. The operator is not liable for failed or delayed uploads.

6. Changes

These terms may be updated. The effective date above shows the latest version.

7. Contact

Contact the operator through the About page of @nunsseulmi_lab.

요약 (한국어)

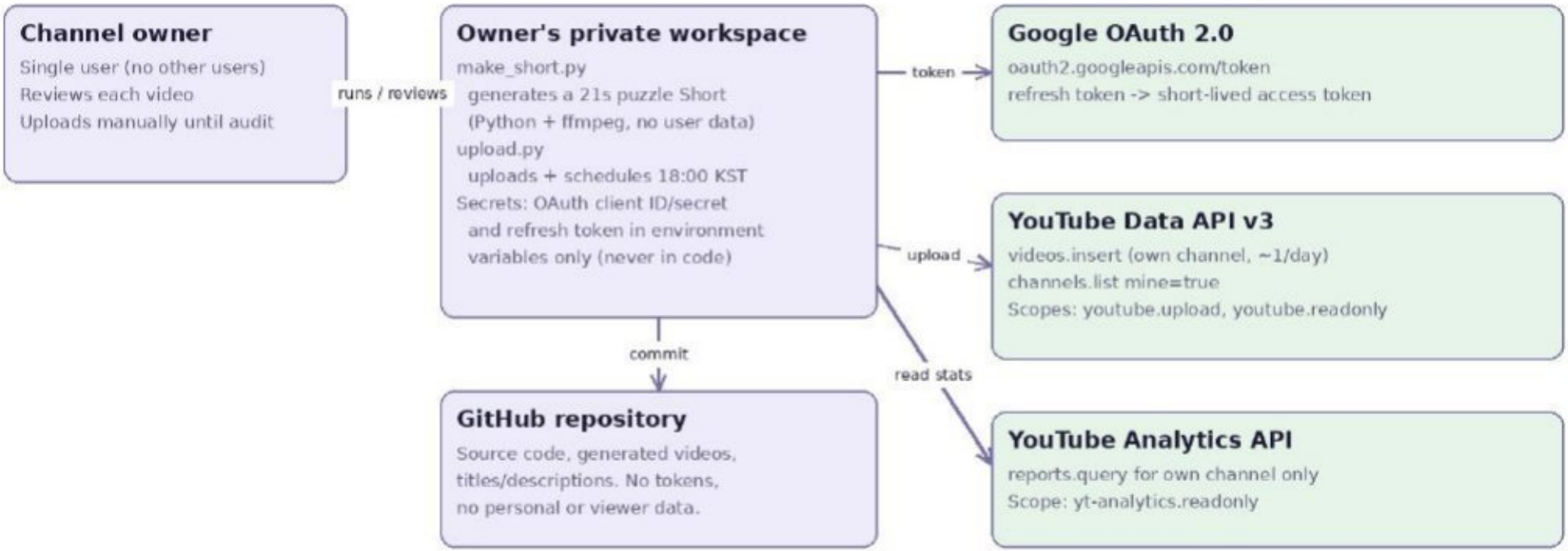
이 도구는 눈썰미 연구소 채널 운영자가 자기 채널에 직접 만든 쇼츠를 올리고 자기 채널 통계를 보는 데만 씁니다. 이 도구는 YouTube API 서비스를 사용하므로 이용자는 YouTube 서비스 약관에 동의하게 되며 Google 개인정보처리방침이 적용됩니다. 정보 처리 방식은 개인정보처리방침을 따릅니다.

4. Architecture diagram

Architecture - Nunsseulmi Lab Uploader

OAuth app name: 눈썰미 연구소 업로더

Private script used only by the owner of YouTube channel @nunsseulmi_lab (UC7DlQBmVmLFcLiquvUfvKLw)



Data handling: only the owner's own channel is accessed. No data is sold, shared, or used for ads.

Nothing about viewers or other channels is collected. Access can be revoked at myaccount.google.com/permissions.

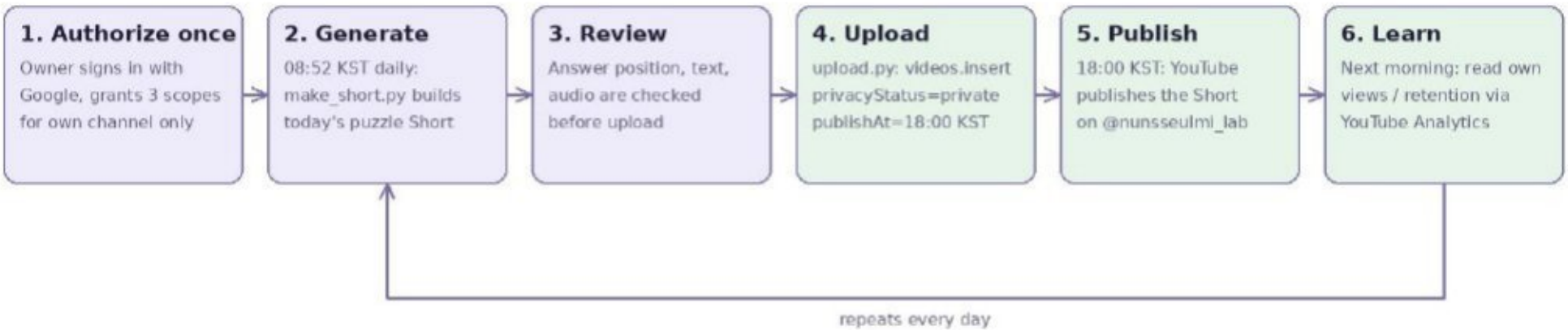
Privacy policy: https://rjgrf1-afk.github.io/Ju_HJ/privacy.html

5. User flow diagram

User flow - Nunsseulmi Lab Uploader

OAuth app name: 눈썰미 연구소 업로더

The only user is the channel owner. Steps 2-6 repeat daily.



OAuth consent: the owner sees the Google consent screen of this app and approves:

youtube.upload (upload own videos), youtube.readonly (list own channel), yt-analytics.readonly (own stats).

The refresh token is stored as a private environment variable of the owner's workspace.

Revoke: myaccount.google.com/permissions -> the script immediately loses access.

Expected volume: about 1 upload and a few read calls per day (well under the default quota).

Project #1 - Conditional evidence

App: 눈썰미 연구소 업로더 (Nunsseulmi Lab Uploader)

Google Cloud project number: 203046541838

Channel: @nunsseulmi_lab (UC7DlQBmVmLFcLiquvUfvKLw)

a) OAuth flow - pages 2-5

unverified-app warning, consent screen, scopes, revocation

b) Upload interface - page 6

upload.py private test upload (videoId TcvL9qNbR6M)

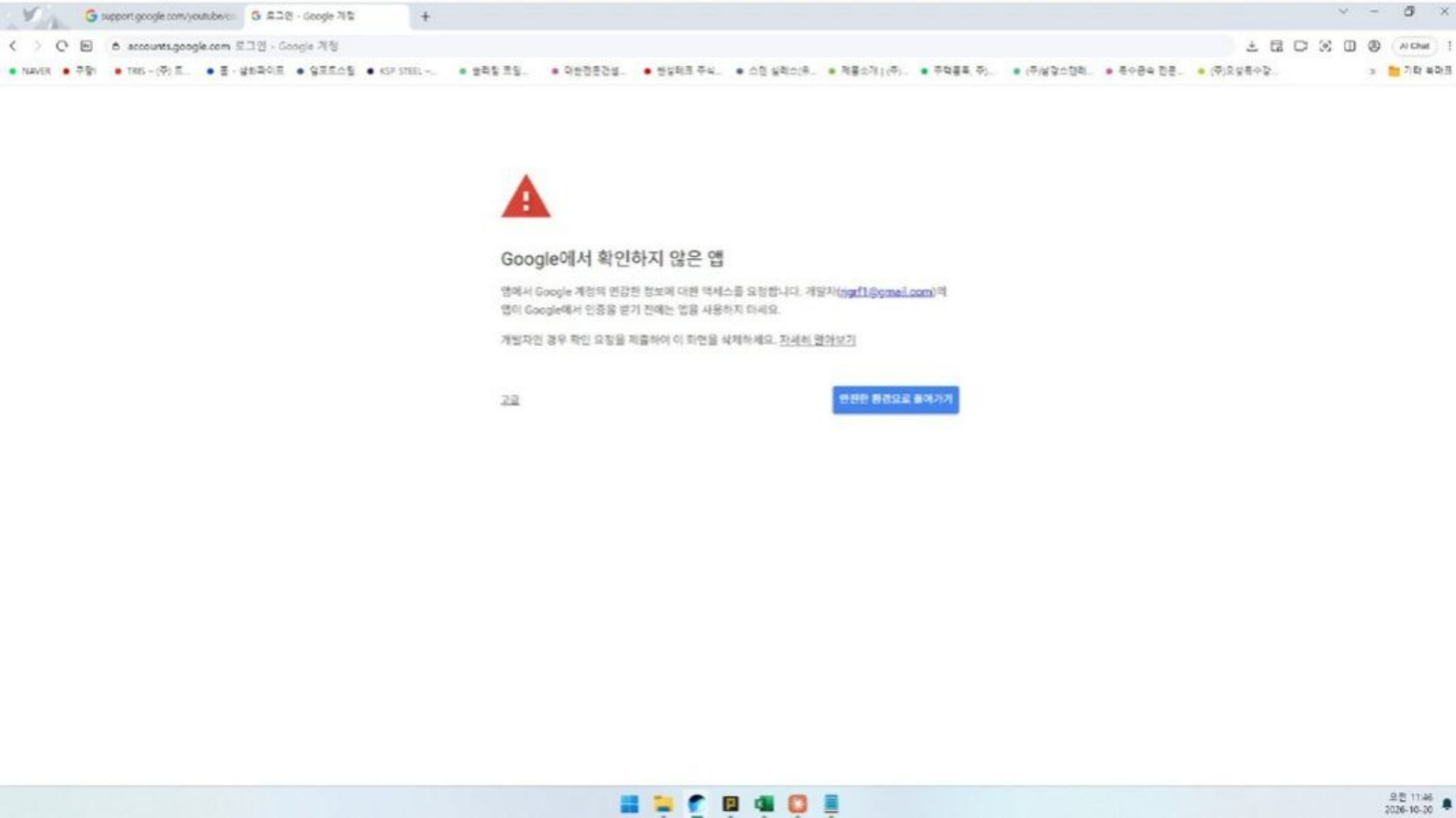
d) Analytics / reporting feature - page 7

channel_report.py reads only the owner's own channel

6. Conditional evidence

Section a) OAuth flow

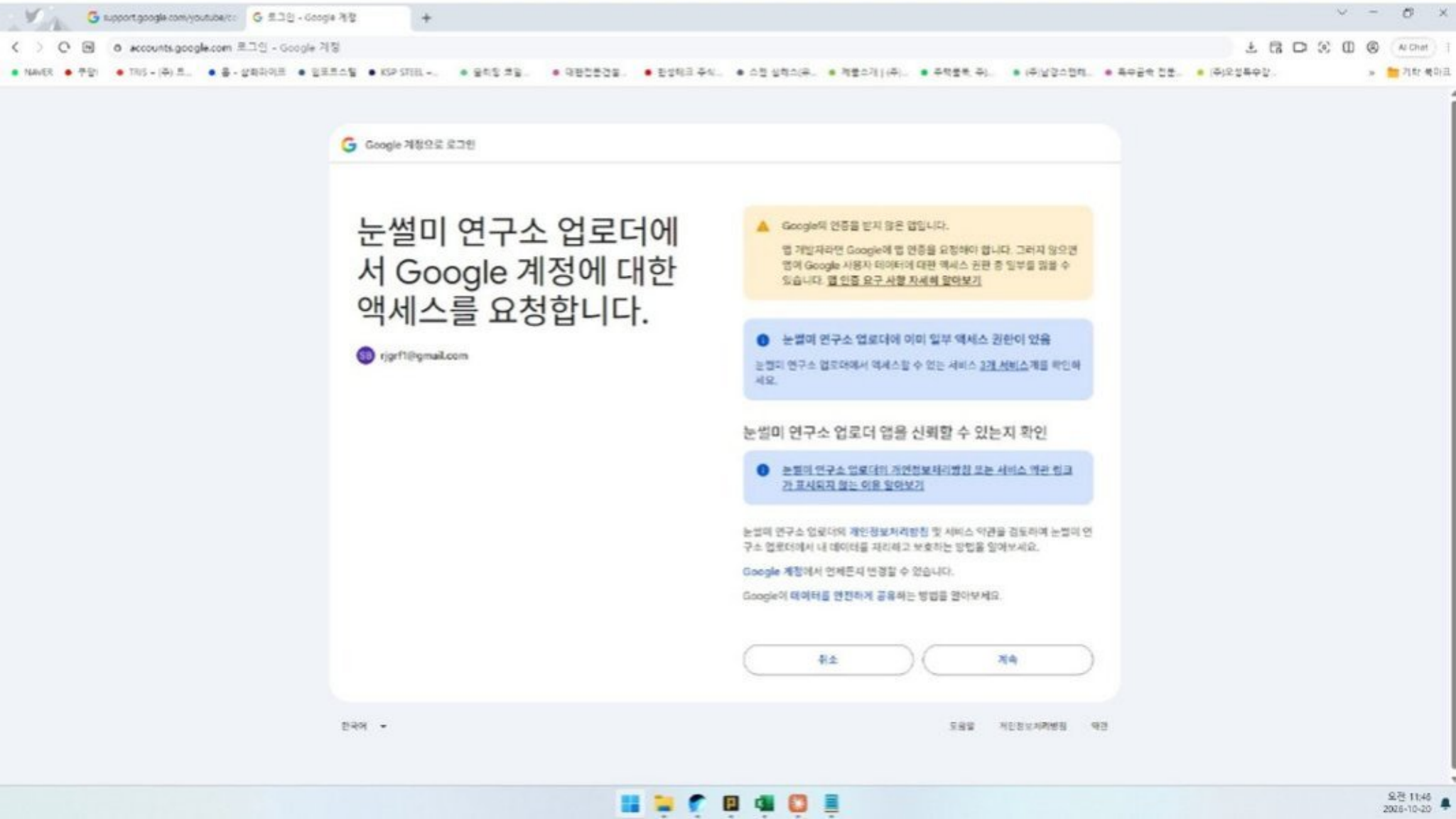
a-1 Unverified-app warning shown before consent



6. Conditional evidence

Section a) OAuth flow

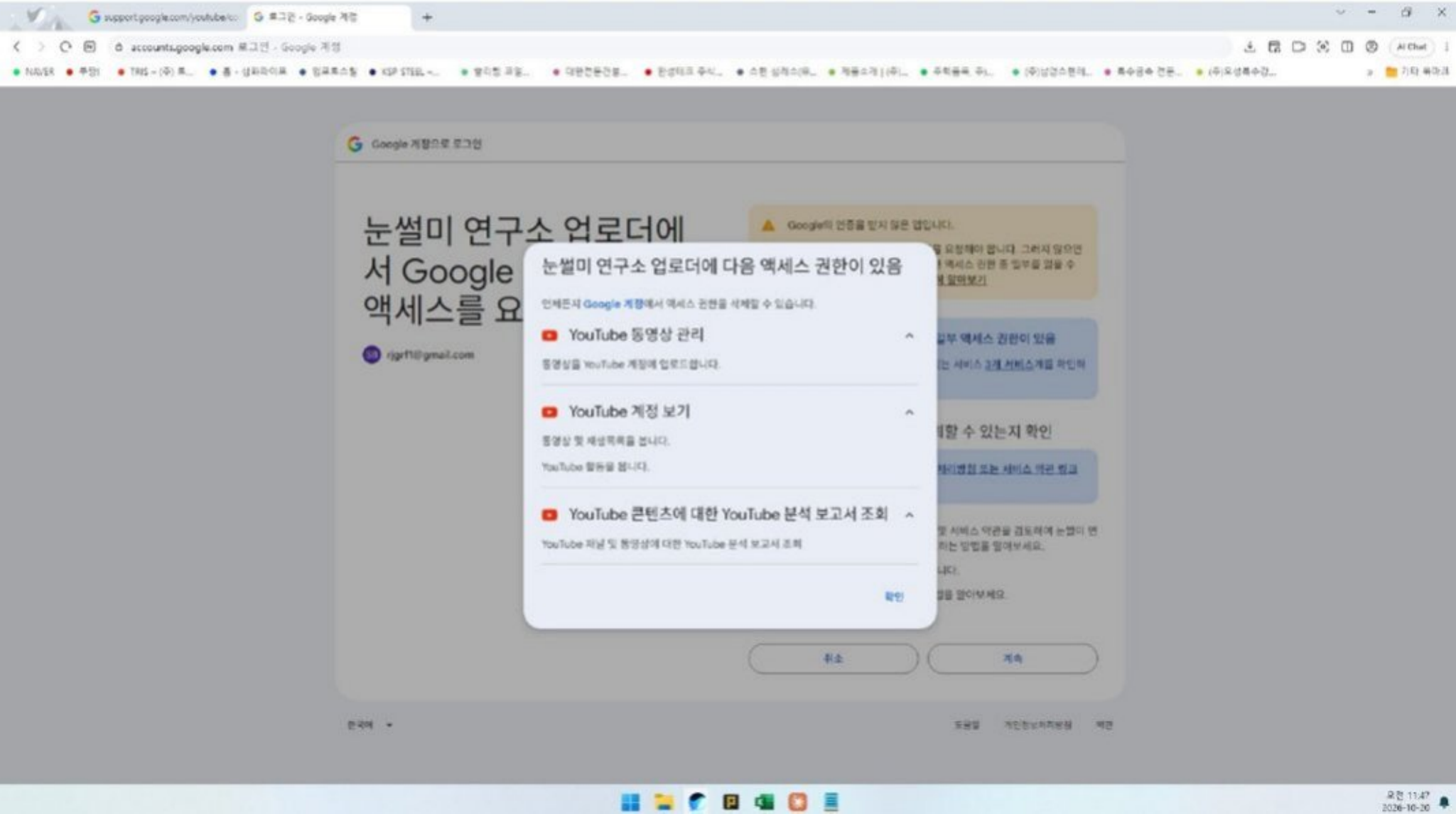
a-2 Consent screen: app name "눈썰미 연구소 업로더", privacy policy and terms



6. Conditional evidence

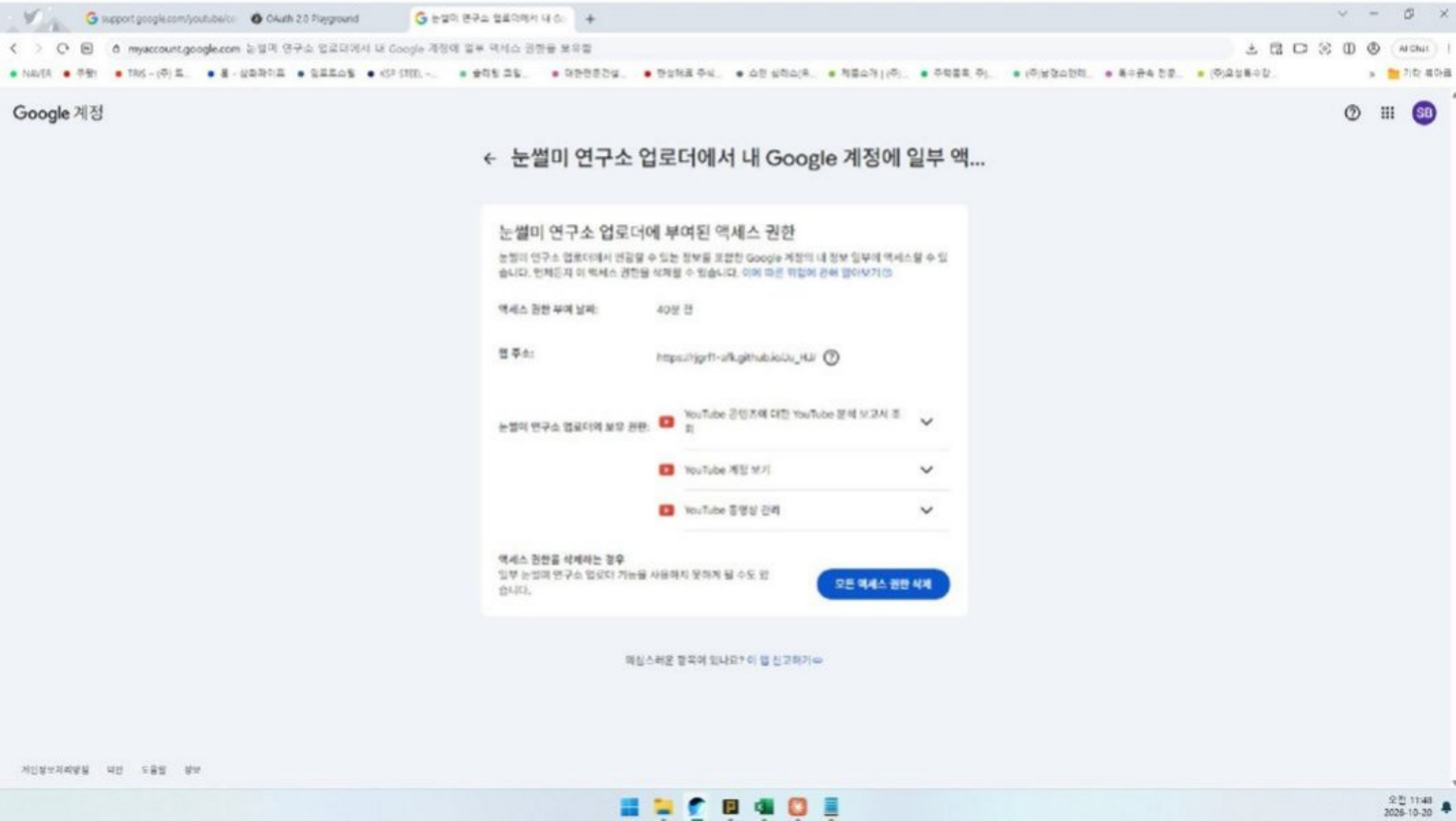
Section a) OAuth flow

a-3 Requested scopes: upload videos, view account, view Analytics reports



Section a) OAuth flow

a-4 Revocation: myaccount.google.com/permissions with "remove all access"



6. Conditional evidence

Section b) Upload interface - uploads the owner's own Short to @nunsseulmi_lab (private test upload)

```
$ python3 shorts/upload.py --date 2026-10-20 --private
업로드 완료: https://youtube.com/shorts/TcvL9qNbR6M (공개: private)

== videos.list(part=status,snippet, id=TcvL9qNbR6M) ==
channelTitle  : 눈썰미 연구소
title         : 다른 하나 찾기 🔍 3단계 성공하면 상위 1% #shorts
privacyStatus : private
uploadStatus  : uploaded
publishAt     : (none)
```

6. Conditional evidence

Section d) Analytics/reporting feature - reads only the owner's own channel

```
$ python3 shorts/channel_report.py
== Channel (YouTube Data API v3 channels.list, mine=true) ==
+--------------+----------------+------------+--------+------------+
|        Title |         Handle | Subscribers | Videos | Total views |
+--------------+----------------+------------+--------+------------+
| 눈썰미 연구소 | @nunsseulmi_lab |          0 |      0 |          0 |
+--------------+----------------+------------+--------+------------+

== Last 28 days 2026-09-23 ~ 2026-10-20 (YouTube Analytics API reports.query) ==
+------------+-------+------------------------+----------------------+------------------+
|        day | views | estimatedMinutesWatched | averageViewPercentage | subscribersGained |
+------------+-------+------------------------+----------------------+------------------+
| 2026-09-23 |     0 |                      0 |                    0 |                0 |
| 2026-09-24 |     0 |                      0 |                    0 |                0 |
| 2026-09-25 |     0 |                      0 |                    0 |                0 |
| 2026-09-26 |     0 |                      0 |                    0 |                0 |
| 2026-09-27 |     0 |                      0 |                    0 |                0 |
| 2026-09-28 |     0 |                      0 |                    0 |                0 |
| 2026-09-29 |     0 |                      0 |                    0 |                0 |
| 2026-09-30 |     0 |                      0 |                    0 |                0 |
| 2026-10-01 |     0 |                      0 |                    0 |                0 |
| 2026-10-02 |     0 |                      0 |                    0 |                0 |
| 2026-10-03 |     0 |                      0 |                    0 |                0 |
| 2026-10-04 |     0 |                      0 |                    0 |                0 |
| 2026-10-05 |     0 |                      0 |                    0 |                0 |
| 2026-10-06 |     0 |                      0 |                    0 |                0 |
| 2026-10-07 |     0 |                      0 |                    0 |                0 |
| 2026-10-08 |     0 |                      0 |                    0 |                0 |
| 2026-10-09 |     0 |                      0 |                    0 |                0 |
| 2026-10-10 |     0 |                      0 |                    0 |                0 |
| 2026-10-11 |     0 |                      0 |                    0 |                0 |
| 2026-10-12 |     0 |                      0 |                    0 |                0 |
| 2026-10-13 |     0 |                      0 |                    0 |                0 |
| 2026-10-14 |     0 |                      0 |                    0 |                0 |
| 2026-10-15 |     0 |                      0 |                    0 |                0 |
| 2026-10-16 |     0 |                      0 |                    0 |                0 |
| 2026-10-17 |     0 |                      0 |                    0 |                0 |
+------------+-------+------------------------+----------------------+------------------+
+---------+-------+------------------------+----------------------+------------------+
|   total | views | estimatedMinutesWatched | averageViewPercentage | subscribersGained |
+---------+-------+------------------------+----------------------+------------------+
| 25 days |     0 |                      0 |                  0.0 |                0 |
+---------+-------+------------------------+----------------------+------------------+
```

7. Complete upload script: shorts/upload.py (page 1/2)

Full source, unmodified. Repository: github.com/rjgrf1-afk/Ju_HJ (branch ccr-aaca7a09-rdvvku)

```
"""만든 쇼츠를 YouTube에 예약 업로드한다 (YouTube Data API v3).

필요한 환경 변수 (README의 '계정 연결' 참고):
    YT_CLIENT_ID, YT_CLIENT_SECRET, YT_REFRESH_TOKEN

사용법:
    python3 shorts/upload.py --date 2026-10-20           # 그날 18:00 KST 공개 예약
    python3 shorts/upload.py --date 2026-10-20 --now     # 바로 공개
    python3 shorts/upload.py --date 2026-10-20 --private  # 비공개 (예약 공개 없음)
"""
import argparse
import json
import os
from datetime import datetime, timedelta, timezone
from pathlib import Path

import requests

ROOT = Path(__file__).resolve().parent
KST = timezone(timedelta(hours=9))
PUBLISH_HOUR = 18


def access_token():
    r = requests.post("https://oauth2.googleapis.com/token", data={
        "client_id": os.environ["YT_CLIENT_ID"],
        "client_secret": os.environ["YT_CLIENT_SECRET"],
        "refresh_token": os.environ["YT_REFRESH_TOKEN"],
        "grant_type": "refresh_token",
    }, timeout=30)
    r.raise_for_status()
    return r.json()["access_token"]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--date", default=datetime.now(KST).strftime("%Y-%m-%d"))
    vis = ap.add_mutually_exclusive_group()
    vis.add_argument("--now", action="store_true", help="예약 없이 바로 공개")
    vis.add_argument("--private", action="store_true", help="비공개로만 올림 (예약 공개 없음)")
    args = ap.parse_args()

    out = ROOT / "output" / args.date
    meta = json.loads((out / "metadata.json").read_text())
    if meta.get("videoId"):
        raise SystemExit(f"이미 업로드됨: https://youtube.com/shorts/{meta['videoId']}")

    status = {
        "selfDeclaredMadeForKids": meta["madeForKids"],
        "containsSyntheticMedia": meta["containsSyntheticMedia"],
    }
    if args.now:
        status["privacyStatus"] = "public"
    elif args.private:
        status["privacyStatus"] = "private"
    else:
        publish = datetime.strptime(args.date, "%Y-%m-%d").replace(hour=PUBLISH_HOUR, tzinfo=KST)
        status["privacyStatus"] = "private"  # publishAt은 private일 때만 동작
        status["publishAt"] = publish.astimezone(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")

    body = {
        "snippet": {
            "title": meta["title"],
            "description": meta["description"],
            "tags": meta["tags"],
```

7. Complete upload script: shorts/upload.py (page 2/2)

Full source, unmodified. Repository: github.com/rjgrf1-afk/Ju_HJ (branch ccr-aaca7a09-rdvvku)

```
            "categoryId": meta["categoryId"],
            "defaultLanguage": "ko",
        },
        "status": status,
    }
    video = (out / "short.mp4").read_bytes()
    headers = {"Authorization": f"Bearer {access_token()}"}

    init = requests.post(
        "https://www.googleapis.com/upload/youtube/v3/videos",
        params={"uploadType": "resumable", "part": "snippet,status"},
        headers={**headers, "X-Upload-Content-Type": "video/mp4",
                 "X-Upload-Content-Length": str(len(video))},
        json=body, timeout=60)
    init.raise_for_status()
    up = requests.put(init.headers["Location"], data=video,
                      headers={**headers, "Content-Type": "video/mp4"}, timeout=600)
    up.raise_for_status()

    meta["videoId"] = up.json()["id"]
    meta["publishAt"] = status.get("publishAt", "private" if args.private else "now")
    (out / "metadata.json").write_text(json.dumps(meta, ensure_ascii=False, indent=2))
    print(f"업로드 완료: https://youtube.com/shorts/{meta['videoId']} (공개: {meta['publishAt']})")


if __name__ == "__main__":
    main()
```